Coherence Governance for LLM Conversations

A Four-Layer Architecture with State Discipline

Technical Specification — v1.0

Hanns-Steffen Rentschler

February 14, 2026

An epistemically grounded coherence architecture for stateless language models

---

Core Thesis

A system without a meta-layer cannot know when it is wrong.
A system without layer separation does not know what it is changing.

Abstract

Large Language Models (LLMs) process conversations as unstructured text streams without explicit state. As a result, coherence in long dialogues is a statistical byproduct rather than a guaranteed system property. Common failure modes include silent overwriting of earlier statements, semantic drift of definitions and constraints, and the production of superficially consistent but internally contradictory outputs (coherence theater).

This paper introduces Coherence Governance, a four-layer architecture that makes coherence operationally enforceable through explicit state transitions and mandatory validation gates. The architecture separates the processing pipeline into CAPTURE (epistemic classification of inputs), VALIDATE (state-level coherence checking), SEAL (explicit state transitions via a minimal Patch DSL), and RENDER (context-clean output generation). A strict top-down invariant ensures that downstream layers cannot decide what upstream layers have not permitted.

VALIDATE is modeled as a risk controller rather than a truth oracle, combining deterministic checks, semantic entailment tests, and adversarial validation to detect conflicts and uncertainty. State changes are performed exclusively through explicit patch operations (add, modify, deprecate, branch), preventing silent overwriting and enabling auditable conversation histories. A multi-tier seal policy balances stability and flexibility, while a compact chat syntax makes the framework usable in everyday dialogue.

The proposed framework does not aim to guarantee factual correctness or prevent malicious input. Instead, it enforces epistemic discipline: contradictions become visible, uncertainty is surfaced, and coherence failures are contained through branching rather than implicit drift. Coherence Governance is implementable as a wrapper around existing LLMs and complements, rather than replaces, memory augmentation and guardrail approaches.

---

0 Executive Summary

Current Large Language Models (LLMs) process conversations as text streams without explicit state. Coherence over longer dialogues is a statistical byproduct, not a guaranteed system property. This document describes a four-layer architecture that makes coherence enforceable rather than hope-based.

The architecture separates the processing pipeline into four sequential layers: CAPTURE (input classification), VALIDATE (coherence checking against existing state), SEAL (binding state changes via Patch DSL), and RENDER (context-clean output). This is complemented by a formal Patch DSL for traceable state changes, an operational decision table for the validator, a three-tier seal policy, and a practical chat mode for everyday use.

Fundamental Invariant: Nothing downstream may decide what upstream has not permitted. RENDER may only speak from the currently active state, and the state may only change through explicit patches.

1 Problem Statement

LLMs suffer from systematic coherence loss in long conversations. Earlier statements are forgotten or treated inconsistently, tone drifts, and initial instructions lose influence. The root cause is architectural: LLMs reconstruct the entire state from the text stream on every turn, without persistent state management.

1.1 Failure Classes

Three failure classes dominate:

- **Silent Overwriting:** A new statement implicitly replaces an earlier one without the contradiction being detected or documented.
- **Coherence Theater:** The model produces eloquent but internally contradictory outputs that appear superficially consistent.
- **Layer Conflation:** Facts, hypotheses, stylistic decisions, and rules are not distinguished and compete undifferentiated for influence.

1.2 Why Existing Approaches Fall Short

Memory-Augmented LLMs (MemoryBank, GOLF) address coherence through retrieval and forgetting curves, not through explicit state validation. The Structured Cognitive Loop (Kim, 2025) introduces a governance layer for agent actions but not for epistemic coherence within a conversation. Guardrails frameworks (NeMo, Guardrails AI) filter input/output for safety, not for content consistency across turns.

The Gap: No existing approach treats coherence within a single chat as a standalone governance problem with epistemic layer separation and formal state control.

2 Four-Layer Architecture

Processing flows strictly top-down. Each layer has a clearly defined task and a failure class it prevents.

2.1 Layer I — CAPTURE (Input Layer)

Task: Classification, not interpretation. Every new statement is sorted into an epistemic layer.

Epistemic Layers

Label	Layer	Description
F	Facts / Canon	Secured, binding statements

D	Definitions	Term definitions that serve as anchors
R	Rules / Constraints	Restrictions that govern system behavior
H	Hypotheses	Working hypotheses, exploratory, revocable
S	Style / Rhetoric	Tonality, formatting, presentation preferences
M	Meta-Instructions	Governance level: how the system should operate

Rule: Nothing in this layer may alter existing content. CAPTURE is pure decoding and typing.

Prevented failure class: *That sounded plausible, so I just adopted it.*

2.2 Layer II — VALIDATE (Meta-Layer & Coherence Check)

VALIDATE operates not on content but on state. It checks whether new inputs are compatible with the existing state.

Meta-State (mandatory before any processing)

- **Current Mode:** analysis | creative | hybrid
- **Priority Order:** User > Canon > Derivation > Style
- **Conflict Strategy:** stop | branch | prefer-higher-priority
- **Commit Rule:** auto | after OK | after chapter

Mandatory Checks

- **Layer Collision:** Does a new statement attack a higher layer?
- **Canon Violation:** Contradiction with sealed F or D?
- **Constraint Check:** Does something violate R?
- **Scope Check:** Does this apply locally or globally?

The result is never text but a decision: OK | CONFLICT | AMBIGUOUS | REQUIRES USER DECISION

Prevented failure class: *I elegantly continued writing even though I was actually in contradiction.*

2.3 Layer III — SEAL (Commit, Patch, or Branch)

This is where binding state changes happen. Three operations are permitted:

- **PATCH:** Targeted modification with justification of an existing entry.
- **SEAL:** Closing a scope — afterwards only modifiable via Patch.
- **BRANCH:** Clean fork of a variant when contradictions need to coexist.

Absolute Rule: No silent overwriting. No implicit forgetting.

Prevented failure class: *Earlier you said X, now you say Y, without noticing.*

2.4 Layer IV — RENDER (Output)

Only in this layer is language generated. RENDER is a projection of the current state, not a thinking phase.

- **Access only to the current, validated state.**
- **Every output knows its mode:** analytical | narrative | hypothetical
- **Uncertainty is explicitly marked,** not rhetorically concealed.

RENDER may not generate new facts, only present, combine, and formulate.

Prevented failure class: *The answer sounds good, but it invents new truths along the way.*

2.5 Processing Order (Top-Down Invariant)

The order is not optional but constitutive:

- **1. META first:** Mode, priorities, conflict strategy are fixed before any content access.
- **2. Rules & Definitions before content:** R and D act as validators, not text blocks.
- **3. Canon before derivation:** F (sealed facts) are read-only. Everything new must measure up.
- **4. Hypotheses last:** H may be flexible, but only within the permitted scope.
- **5. RENDER at the very end:** Output is projection, not a thinking phase.

3 VALIDATE in Detail: Multi-Channel Validation with Escalation

VALIDATE is not a Truth Oracle but a Risk Controller. The central insight: VALIDATE does not need to know what is true. It needs to reliably know when it is uncertain.

3.1 Three Validation Tiers

Tier A: Hard Checks (deterministic, cheap)

- **Key/Value Invariants:** When canon exists as structured claims (Subject–Predicate–Object + Scope), identical slots can be checked.
- **Scope and Time Collisions:** Constraints like valid-since-X, only-in-chapter-3, Variant-B are formally checkable.
- **Constraint Violations:** Rules like no-modern-technology are keyword/pattern/ontology-checkable.

Result: Hard conflicts + suspicion flags.

Tier B: Semantic Similarity + Entailment (robust, not absolute)

Contradiction is not defined as LLM intuition but as an entailment decision with an uncertainty budget:

- Canon claim c and new statement n are paraphrased into multiple normal forms.
- Check: $c \Rightarrow n$? $n \Rightarrow c$? $c \perp n$? (Contradiction / Independence)
- Multi-pass redundancy (k consistency checks) + worst-case policy.

Tier C: Adversarial Validator (Anti-Theater)

A second process deliberately attempts to find contradictions:

- Under which interpretations does n contradict the canon?
- What silent assumptions would n need to be compatible?
- Provide a minimal counterexample.

If C finds something, it does not correct but branches or stops.

3.2 Evidence Definition

Strong Evidence (at least one condition met):

- Hard check finds slot collision with opposing value and overlapping scope.
- Adversarial Validator produces a minimal counterexample without additional assumptions.
- Multi-pass check ($k \geq 3$) consistently yields Contradiction.

Weak Evidence:

- Only 1–2 checkers flag conflict; others are uncertain.
- Conflict depends on implicit assumptions (timing, exception, definition).

Policy: With Weak Evidence, OK is never given — FLAG or BRANCH depending on Impact.

Anti-Theater Definition: VALIDATE is good when it rarely lets a real conflict pass — even at the cost of too many warnings.

VALIDATE is intentionally conservative. Its objective is to minimize false negatives (undetected contradictions), even at the cost of increased branching or flagging.

4 Decision Table (Operational)

4.1 Input Variables

Variable	Values	Meaning
T (Claim Type)	F, D, R, H, S	Epistemic layer of the statement
Seal	none, soft, hard, milestone	Current seal status
Scope	local, block, global	Scope of validity
Impact	low, med, high	Number of dependent claims
Match	none, same-key, related	Relation to existing canon

Contradiction Evidence	none, weak, strong	Result of validation tiers
ConfidenceBand	low, med, high	Validator certainty
ConflictMode	stop, branch, prefer	Configured conflict strategy
RiskBudget	strict, balanced, loose	Configured risk tolerance

4.2 Output Actions

Action	Meaning
OK	Integrate without conflict
FLAG	Integrate but mark as uncertain; logged in P
BRANCH	Create variant
STOP	Do not integrate; clarification needed
REQUIRE_USER	Explicit user decision required
PATCH_REQUIRED	Only permitted via Patch on sealed containers

4.3 Priority and Lock Rules (before any table lookup)

A1: Sealed Lock

If Seal is hard or milestone and the operation would modify F/D/R: output is at minimum PATCH_REQUIRED. With additional ContradictionEvidence of weak or strong: STOP or BRANCH depending on ConflictMode.

A2: Container Hierarchy

- **R violated:** Never silently OK. At minimum FLAG, often STOP.
- **D drifting:** Prefer STOP or PATCH_REQUIRED.
- **F contradicts sealed F:** STOP/BRANCH, never overwrite.

4.4 Core Table B1: No Canon Reference (Match = none)

T	Seal	Impact	Decision
H	any	low/med	OK (in H, Scope = local/block)
H	any	high	FLAG (in H + P: high impact)
F	soft/none	low/med	FLAG → as proposed fact, not sealed
F	hard/ms	any	REQUIRE_USER
D/R	any	any	REQUIRE_USER
S	any	any	OK

4.5 Core Table B2: Direct Conflict (Match = same-key)

Evidence	Confidence	Seal	Decision
none	high	any	OK
none	med/low	any	FLAG
weak	high/med	soft/none	FLAG or BRANCH
weak	any	hard/ms	PATCH_REQ + STOP
strong	any	soft/none	BRANCH or STOP
strong	any	hard/ms	PATCH_REQ + STOP

4.6 Core Table B3: Semantic Conflict (Match = related)

This is the most common failure source — this is where LLMs typically perform coherence theater.

Evidence	Confidence	Impact	Decision
none	high	low/med	OK
none	med/low	any	FLAG
weak	high/med	low	FLAG (note: tension)
weak	any	med/high	BRANCH or REQ_USER
strong	any	any	BRANCH or STOP

4.7 Special Rules by Claim Type

- **Type R (Rules):** Any rule collision triggers at minimum STOP or REQUIRE_USER. New rules without explicit mandate: REQUIRE_USER.
- **Type D (Definitions):** Definition drift is almost always systemic → STOP/PATCH_REQUIRED. Only local definitions may be OK.
- **Type F (Facts):** New facts enter as proposed fact (Soft-Seal) unless the user says Canon. Contradiction with sealed canon: never auto-override.
- **Type H (Hypotheses):** May coexist contradictorily as long as they remain separated (labels H1/H2 or BRANCH at high impact).

5 Seal Policy

5.1 Three Seal Tiers

Tier	Trigger	Effect
Soft-Seal	Default after every turn	Turn is recorded in H/P, nothing immutable
Hard-Seal	Topic block / User OK	F/D/R become read-only, changes only via Patch
Milestone-Seal	Explicit fixed points	Major sections that must not drift

5.2 Triggers for Hard-Seal

- **User Trigger:** Fix, Canon, from now on this applies, OK confirmed
- **Stability Trigger:** Same claim used k times consistently without conflict (e.g. k = 3)
- **Cost Trigger:** Later changes would be expensive (many dependent statements)

5.3 Countermeasure Against Rigidity

Patching has a cost but is always possible. The system produces stability without dogma. Deprecate instead of Delete: history remains visible.

6 Meta-Governance (M0 / M1)

META rules are patchable — but only with an unpatchable kernel layer. Without this, the system can arbitrarily weaken its own constitution.

6.1 M0: Kernel (immutable)

- **Priority Axioms:** User corrections override model derivations.
- **Safety/Integrity Rules:** No silent overwriting. Log everything as patches.
- **Mandatory Steps:** CAPTURE → VALIDATE → SEAL → RENDER

6.2 M1: Policy (patchable)

- Conflict strategy (branch vs stop)
- Seal granularity
- Tolerance thresholds (how aggressively VALIDATE flags)
- Style/output preferences

Rule: Patching M1 itself runs through M0. VALIDATE checks whether the M1 patch violates the kernel. Self-reference is possible but not anarchic.

7 Patch DSL

Without a Patch DSL, VALIDATE remains a warning system without state discipline. The DSL makes every state change an explicit, traceable, reversible act.

7.1 Design Goals

- No silent overwriting: Every state change is a patch.
- Four operations: `add` | `modify` | `deprecate` | `branch`
- Mandatory fields enforce traceability.
- Seal-respecting: Sealed containers only via Patch, never implicitly.

7.2 Patch Envelope

Every patch has a mandatory header:

Field	Format	Description
id	P-YYYY-MM-DD-NNN	Sequential identifier
actor	user | assistant	Who is patching
mode	analysis | creative | hybrid	Current mode
reason	1 sentence	Justification (not marketing)

7.3 Mandatory Fields per Operation

Field	Values	Description
op	add | modify | deprecate | branch	Operation type
container	F, D, R, H, P, S, M1	Target container (M0 not patchable)
key	subject::predicate::object#qual	Canonical identifier
scope	local | block:<n> | global	Scope of validity
seal	none | soft | hard | milestone	Target seal status after patch
evidence	observation | derivation | ...	Evidence type
confidence	low | med | high	Validator confidence

7.4 Operations in Detail

ADD

Adds a new entry. Additional fields: `value` (the value) and `status` ∈ {proposed, active}. Default for F/D/R: proposed, until hard-sealed.

Rule: If key exists and is active: add only permitted with container = H (coexisting hypotheses) or branch scope.

MODIFY

Changes an existing entry. Additional fields: `from`, `to`, `delta` (change reason), `compat` ∈ {narrowing, broadening, correction, reframe}.

Rule: With seal = hard | milestone: modify must specify conflicts or depends_on, otherwise STOP.

compat types: Force classification of the change type. Prevents corrections from being disguised as refinements.

DEPRECATE

Invalidates without deleting. Additional fields: `deprecated_by`, `replacement_key` (optional), `sunset` ∈ {immediate, after:block:<n>}.

Principle: Deprecate instead of Delete is the anti-theater hammer: history remains visible.

BRANCH

Creates a new fork. Additional fields: `branch_name`, `diverges_at`, `copy_policy` ∈ {shallow, deep}.

Rule: Branch is always permitted, even on milestone-sealed canon, because it does not overwrite anything.

7.5 Key Format

Recommended format: Triple + Qualifier as string ID.

```
<subject>::<predicate>::<object_or_slot>#<qualifiers>

Examples:
Vico::birth_status::recorded_dead#world=roman;time=1637
Style::avoid_word::shadow#scope=global
Coherence::pipeline::order#context=chat
```

7.6 The Load-Bearing Axiom

RENDER may only speak from the currently active state. And the state may only change through patches. This effectively kills coherence theater, because any drift would be visible as a patch.

The state is formally defined as the ordered set of active entries across containers {F, D, R, H, P, S, M1} within a branch.

8 Practical Chat Mode

Two shorthand commands make the framework usable in live dialogue without YAML or architecture text.

8.1 @META — Governance in One Line

```
@META mode=<analysis|creative|hybrid>
      conflict=<stop|branch|prefer>
      seal=<soft|hard|milestone|manual>
      risk=<strict|balanced|loose>
```

8.2 @PATCH — Shorthand Form

Operators: + (add), ~ (modify), - (deprecate), ^ (branch)

```
@PATCH <op> <container> <key> = <value> | opts...

Examples:
@PATCH + R no_silent_overwrite = true | seal=hard
@PATCH + D VALIDATE = "Risk Controller" | seal=hard
@PATCH ~ F pipeline_order = "new value" | seal=hard ev=user
@PATCH - F pipeline_order | repl=pipeline_order_v2
@PATCH ^ P VariantB | at=pipeline_order copy=shallow
```

8.3 Inline Markers (without Patch syntax)

For natural speech with epistemic marking:

Marker	Example	Effect
@F <statement>	@F The earth is round	Internally patched as proposed fact
@R <statement>	@R No passive constructions	Internally patched as rule
@H <statement>	@H Maybe X is the case	Stored as hypothesis
@SEAL F,D,R	@SEAL F,D,R scope=global	Hard-seal all active F/D/R
Canon is fixed now	(natural language)	Equivalent to @SEAL

8.4 Assistant Response (compact)

At the end of each response, a state-delta line appears only when something has changed:

```
StateD: +R no_silent_overwrite (hard); +F pipeline_order (soft)

On conflicts:
VALIDATE: conflict with F:xxx -> BRANCH VariantB
VALIDATE: needs decision -> STOP (ask)
```

8.5 Recommended Startup Configuration

```
@META mode=analysis conflict=branch seal=manual risk=strict
@PATCH + R no_silent_overwrite = true | seal=hard
@PATCH + R sealed_requires_patch = true | seal=hard
```

From here on, the system runs with active governance.

9 Robustness Criteria

The system is robust when it rarely breaks canon unnoticed in long dialogues and instead becomes visible under uncertainty.

9.1 Measurable Criteria

Metric	Definition
Conflict Recall	Proportion of real conflicts detected by VALIDATE
Patch Discipline	Proportion of state changes explicitly patched rather than implicitly made
Branch Cleanliness	Proportion of cleanly separated variants (no contamination between branches)
False Negative Rate	Real conflicts waved through (minimize, even at the cost of more flags)

Robustness does not mean always correct, but: uncertainty becomes visible rather than concealed.

10 Related Work and Differentiation

Approach	Focus	Differentiation
Structured Cognitive Loop (Kim 2025)	Governance for agent actions (tool calls, policies)	Not epistemic; no F/D/R/H layer separation
MemoryBank, GOLF et al.	Memory augmentation via retrieval + forgetting curves	No state validation, no patch system
NeMo Guardrails	Input/output filtering for safety + tonality	No content coherence across turns
Ontology-backed Context (OWL)	Persistent memory for code generation	Domain-specific, no general chat governance
MOLES (Map of LLM Epistemic Stances)	Taxonomy of epistemic stances in LLM output	Classification only, no state control

Novel contribution of this framework: Epistemic layer separation as a governance primitive, top-down validation before every output, explicit state operations (PATCH/SEAL/BRANCH) as conversation version control, operationalized through an everyday chat syntax.

11 Conclusion

This document describes a complete architecture for coherence governance in LLM conversations. The central thesis: coherence must not be a statistical byproduct but must be enforced through explicit state control.

The framework is implementable: the decision table is deterministically decidable, the Patch DSL has a formal syntax, and the chat mode reduces the entry barrier to three lines. The next step is a reference implementation — a wrapper that realizes the three VALIDATE tiers as a pipeline and maintains state as a structured store.

The fundamental difference between context-as-text-stream and context-as-state-with-governance is not theoretical — it is the difference between hope-based and enforceable coherence.

Appendix A — Threat Model & Non-Goals

A.1 Purpose of the Threat Model

This framework addresses epistemic coherence losses arising from missing state management, layer conflation, and implicit overwriting in LLM conversations. It does not claim to prevent all failure modes of language models. This section explicitly names the boundaries.

A.2 Threats Addressed

The framework is explicitly designed to counter the following classes:

- **Silent overwriting** of earlier statements without documentation
- **Semantic drift** of definitions, rules, or canon
- **Coherence theater** (superficially consistent, internally contradictory answers)
- **Uncontrolled context conflation** between facts, hypotheses, and style
- **Implicit decision-making** during the output phase (RENDER)

These threats are structurally addressed through top-down governance, explicit patch operations, and mandatory validation steps.

A.3 Explicit Non-Goals (Unsolved Problems)

The framework deliberately does not solve:

1. Intentional Hallucinations

If a model deliberately introduces false content as fact and correctly patches it as F (e.g., with confidence=high), the framework cannot guarantee its factual truth.

The system enforces traceability, not truth.

2. Malicious User Patches

A user can intentionally hard-seal false rules, definitions, or facts.

The framework protects against implicit errors, not against authoritatively set ones.

3. Faulty Validation Heuristics

Semantic contradiction detection (VALIDATE Tier B/C) remains probabilistic.

The goal is conflict recall, not perfect classification.

4. Cross-Conversation Coherence

State is deliberately conversation-local. Persistent knowledge maintenance across sessions is explicitly out of scope.

5. Moral or Normative Correctness

The framework makes no ethical judgments about content.

It regulates coherence, not values.

A.4 Safety Philosophy

The framework follows a Fail-Loud principle:

It is better to flag or branch too many conflicts than to silently wave through a real contradiction.

Robustness is measured not by always being correct, but by making uncertainty visible.

Appendix B — Mini-Walkthrough (Reviewer Example)

B.0 Starting Situation

A chat about the architecture of LLM coherence. No canon has been set yet.

B.1 Turn 1 — Initialize Governance

```
@META mode=analysis conflict=branch seal=manual risk=strict
```

Effect: META policy is set. No content changes.

B.2 Turn 2 — Fix a Rule

User:

```
@PATCH + R no_silent_overwrite = true | seal=hard
```

System response:

```
VALIDATE: OK
SEAL: Hard-Seal on rule
StateD: +R no_silent_overwrite (hard)
```

B.3 Turn 3 — Introduce a Fact (proposed)

User:

```
@F LLMs lose coherence because they have no explicit state.
```

System response:

```
Internal mapping: add F (status=proposed, seal=soft)
VALIDATE: no canon reference -> OK
StateD: +F coherence_cause_no_state (soft, proposed)
```

B.4 Turn 4 — Contradictory Statement

User:

```
@F Modern LLMs implicitly do have a stable state.
```

VALIDATE evaluation:

Variable	Value
Match	related
Evidence	weak (semantic tension)
Impact	high
RiskBudget	strict

Decision: BRANCH

```
VALIDATE: semantic conflict with F:coherence_cause_no_state -> BRANCH
VariantB
StateD: ^BRANCH VariantB
```

B.5 Turn 5 — Continue in Main Branch

Assistant (in the original branch):

Under the assumption of a stateless model, the coherence loss is architecturally explained ...

(RENDER accesses only the active state.)

B.6 Turn 6 — Fix Canon

User:

```
@SEAL F scope=global
```

Effect: All active facts in the main branch are hard-sealed. The alternative interpretation remains isolated in VariantB.

B.7 Result

- **No silent overwriting:** Every state change is documented.
- **Contradiction made visible:** The conflict was branched, not suppressed.
- **Fully auditable:** A reviewer can trace every state step.
- **Coherence is enforced,** not merely asserted.

B.8 Why These Two Appendices Matter

- **Threat Model** protects against overclaim accusations from reviewers.
- **Walkthrough** demonstrates that the system is operational, not merely conceptual.

Together, they make the paper review-robust.